AI ASSISTED CODING

ASSIGNMENT – 16.1

NAME: Varshitha. G

HTNO: 2303A51103

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:o Insert a new student record.o Fetch all courses enrolled by a student.o Count number of students in each course.

```
# create a tables students, courses, and enrollments and Define primary keys, fore
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
CREATE TABLE enrollments (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
# Insert a new student record into the students, course TABLE
INSERT INTO students (name, age) VALUES ('David', 22);
INSERT INTO courses (name) VALUES ('Mathematics');
INSERT INTO students (name, age) VALUES ('Eve', 28);
INSERT INTO courses (name) VALUES ('Physics');
INSERT INTO students (name, age) VALUES ('Frank', 24);
INSERT INTO courses (name) VALUES ('Chemistry');
INSERT INTO students (name, age) VALUES ('Grace', 26);
INSERT INTO courses (name) VALUES ('Biology');
# Enroll the student in the course
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);
INSERT INTO enrollments (student_id, course_id) VALUES (2, 1);
INSERT INTO enrollments (student_id, course_id) VALUES (3, 2);
INSERT INTO enrollments (student_id, course_id) VALUES (4, 3);
```

```sql
# display all rows from students, courses, and enrollments tables
SELECT * FROM students;
SELECT * FROM courses;
SELECT * FROM enrollments;
# count the number of students enrolled in each course
SELECT courses.name, COUNT(enrollments.student_id) AS student_count
FROM courses
LEFT JOIN enrollments ON courses.id = enrollments.course_id
GROUP BY courses.name;
```

SQL 1

student_name	course_name
Alice	Mathematics
Alice	Physics
Bob	Mathematics
Bob	Physics

SQL

course_name	num_students
Mathematics	2
Physics	2

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

o List all appointments for a specific doctor.

o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

```
CREATE TABLE doctors (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialty VARCHAR(100) NOT NULL
);
CREATE TABLE patients (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE appointments (
    id SERIAL PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES doctors(id),
    FOREIGN KEY (patient_id) REFERENCES patients(id)
);
# Insert some data into the tables
INSERT INTO doctors (name, specialty) VALUES ('Dr. Smith', 'Cardiology');
INSERT INTO doctors (name, specialty) VALUES ('Dr. Johnson', 'Neurology');
INSERT INTO patients (name, age) VALUES ('Alice', 30);
INSERT INTO patients (name, age) VALUES ('Bob', 25);
INSERT INTO appointments (doctor_id, patient_id, appointment_date) VALUES (1, 1, '20
INSERT INTO appointments (doctor_id, patient_id, appointment_date) VALUES (1, 2, '20
INSERT INTO appointments (doctor_id, patient_id, appointment_date) VALUES (2, 1, '20
# List all appointments for a specific doctor (e.g., Dr. Smith)
SELECT appointments.id, patients.name AS patient_name, appointments.appointment_date
FROM appointments
JOIN patients ON appointments.patient_id = patients.id
WHERE appointments.doctor_id = 1;
```

```sql
# Retrieve patient history by patient ID (e.g., patient ID 1)
SELECT appointments.id, doctors.name AS doctor_name, appointments.appointment_date
FROM appointments
JOIN doctors ON appointments.doctor_id = doctors.id
WHERE appointments.patient_id = 1;
# Count total patients treated by each doctor
SELECT doctors.name AS doctor_name, COUNT(DISTINCT appointments.patient_id) AS pati
FROM doctors
LEFT JOIN appointments ON doctors.id = appointments.doctor_id
GROUP BY doctors.name;
```

SQL ▼ 1 / 1

appointment_date	doctor_name	specialty
2023-10-15	Dr. Smith	Cardiology

SQL ▼ 1

appointment_date	patient_name
2023-10-15	Alice
2023-10-16	Bob

SQL ▼

doctor_name	total_patients
Dr. Johnson	1
Dr. Smith	2

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

```sql
CREATE TABLE books (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL
);
CREATE TABLE members (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    membership_date DATE NOT NULL
);
CREATE TABLE loans (
    id SERIAL PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books(id),
    FOREIGN KEY (member_id) REFERENCES members(id)
);
# Create indexes to optimize queries
CREATE INDEX idx_loans_book_id ON loans(book_id);
CREATE INDEX idx_loans_member_id ON loans(member_id);
# Insert some data into the tables
INSERT INTO books (title, author) VALUES ('The Great Gatsby', 'F. Scott Fitzgerald');
INSERT INTO books (title, author) VALUES ('To Kill a Mockingbird', 'Harper Lee');
INSERT INTO members (name, membership_date) VALUES ('Alice', '2024-01-01');
INSERT INTO members (name, membership_date) VALUES ('Bob', '2024-02-01');
INSERT INTO loans (book_id, member_id, loan_date) VALUES (1, 1, '2024-06-01');
INSERT INTO loans (book_id, member_id, loan_date) VALUES (2, 2, '2024-06-15');
# Retrieve all books currently issued (i.e., return_date is NULL)
SELECT books.title, members.name AS member_name, loans.loan_date
FROM loans
JOIN books ON loans.book_id = books.id
JOIN members ON loans.member_id = members.id
WHERE loans.return_date IS NULL;
```

```sql
JOIN books ON loans.book_id = books.id
JOIN members ON loans.member_id = members.id
WHERE loans.return_date IS NULL;
# Find overdue books (loan date > 30 days)
SELECT books.title, members.name AS member_name, loans.loan_date
FROM loans
JOIN books ON loans.book_id = books.id
JOIN members ON loans.member_id = members.id
WHERE loans.return_date IS NULL AND loans.loan_date < DATE('now', '-30 days');
# Count number of books loaned by each member
SELECT members.name AS member_name, COUNT(loans.id) AS books_loaned
FROM members
LEFT JOIN loans ON members.id = loans.member_id
GROUP BY members.name;
```

SQL ▼ 1 / 1 1

title	member_name	loan_date
The Great Gatsby	Alice	2023-10-01

SQL ▼ 1

member_name	total_books_loaned
Alice	1
Bob	1

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

```sql
# create a table customers, items, customer_orders and purchase_items and then 1.
CREATE TABLE customer (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
CREATE TABLE items (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE customer_orders (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customer(id)
);
CREATE TABLE purchase_items (
    id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES customer_orders(id),
    FOREIGN KEY (item_id) REFERENCES items(id)
);
# Insert some data into the tables
INSERT INTO customer (name) VALUES ('Alice');
INSERT INTO customer (name) VALUES ('Bob');
INSERT INTO items (name, price) VALUES ('Laptop', 999.99);
INSERT INTO items (name, price) VALUES ('Phone', 499.99);
INSERT INTO customer_orders (customer_id, order_date) VALUES (1, '2024-06-01');
INSERT INTO customer_orders (customer_id, order_date) VALUES (1, '2024-06-15');
INSERT INTO customer_orders (customer_id, order_date) VALUES (2, '2024-06-20');
INSERT INTO purchase_items (order_id, item_id, quantity) VALUES (1, 1, 1);
INSERT INTO purchase_items (order_id, item_id, quantity) VALUES (1, 2, 2);
INSERT INTO purchase_items (order_id, item_id, quantity) VALUES (2, 1, 1);
INSERT INTO purchase_items (order_id, item_id, quantity) VALUES (3, 2, 1);
# Retrieve all orders by a customer (e.g., customer ID 1)
```

```
INSERT INTO purchase_items (order_id, item_id, quantity) VALUES (3, 2, 1);
# Retrieve all orders by a customer (e.g., customer ID 1)
SELECT customer_orders.id AS order_id, items.name AS item_name, purchase_items.quantity, customer_or
FROM customer_orders
JOIN purchase_items ON customer_orders.id = purchase_items.order_id
JOIN items ON purchase_items.item_id = items.id
WHERE customer_orders.customer_id = 1;
# Find the most purchased item
SELECT items.name AS item_name, SUM(purchase_items.quantity) AS total_quantity
FROM purchase_items
JOIN items ON purchase_items.item_id = items.id
GROUP BY items.name
ORDER BY total_quantity DESC
LIMIT 1;
# Calculate total revenue in a given month (e.g., June 2024)
SELECT SUM(items.price * purchase_items.quantity) AS total_revenue
FROM customer_orders
JOIN purchase_items ON customer_orders.id = purchase_items.order_id
JOIN items ON purchase_items.item_id = items.id
WHERE customer_orders.order_date >= '2024-06-01' AND customer_orders.order_date < '2024-07-01';
# Normalization improvements: The current schema is already normalized to 3NF, as it eliminates redu
# Query optimization: To optimize the queries, we can create indexes on the foreign key columns (
user_id in orders, order_id and product_id in order_details) to speed up join operations. Additional
```

SQL ▾ 1 / 1 1 - 2 of 2

order_id	order_date	product_name	quantity
1	2023-10-01	Laptop	1
1	2023-10-01	Mouse	2

SQL ▾ 1 / 1 1 - 1 of 1

product_name	total_quantity
Mouse	2

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

```sql
CREATE TABLE student_details (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    age INT NOT NULL
);
CREATE TABLE subject_info (
    subject_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);
CREATE TABLE exam_info (
    exam_id INTEGER PRIMARY KEY,
    subject_id INTEGER NOT NULL,
    exam_date DATE NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subject_info(subject_id)
);
CREATE TABLE result_data (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    exam_id INTEGER NOT NULL,
    marks INT NOT NULL,
    FOREIGN KEY (student_id) REFERENCES student_details(student_id),
    FOREIGN KEY (exam_id) REFERENCES exam_info(exam_id)
);
INSERT INTO student_details (name, age) VALUES ('Alice', 20);

INSERT INTO subject_info VALUES (1, 'Math');
INSERT INTO subject_info VALUES (2, 'Science');

INSERT INTO exam_info VALUES (1, 1, '2024-06-01');
INSERT INTO exam_info VALUES (2, 2, '2024-06-02');

INSERT INTO result_data (student_id, exam_id, marks) VALUES (1, 1, 85);
INSERT INTO result_data (student_id, exam_id, marks) VALUES (1, 2, 90);
```

```sql
INSERT INTO subject_info VALUES (1, 'Math');
INSERT INTO subject_info VALUES (2, 'Science');

INSERT INTO exam_info VALUES (1, 1, '2024-06-01');
INSERT INTO exam_info VALUES (2, 2, '2024-06-02');

INSERT INTO result_data (student_id, exam_id, marks) VALUES (1, 1, 85);
INSERT INTO result_data (student_id, exam_id, marks) VALUES (1, 2, 90);

SELECT subject_info.name AS subject_name, AVG(result_data.marks) AS avg_marks
FROM result_data
JOIN exam_info ON result_data.exam_id = exam_info.exam_id
JOIN subject_info ON exam_info.subject_id = subject_info.subject_id
GROUP BY subject_info.name;

SELECT subject_info.name AS subject_name, result_data.marks
FROM result_data
JOIN exam_info ON result_data.exam_id = exam_info.exam_id
JOIN subject_info ON exam_info.subject_id = subject_info.subject_id
WHERE result_data.student_id = 1;
```

subject_name	avg_marks
Math	85.0
Science	90.0

subject_name	marks
Math	85
Science	90